
PROBLEM STATEMENT

In many electronic systems and IoT-based applications, accurate time and date tracking is essential for performing time-dependent operations such as data logging, automation control, and scheduling tasks. Traditional digital clocks and microcontroller-based systems often lose track of time when power is disconnected, leading to inaccuracy and inconvenience.

There is a need for a simple, reliable, and low-cost system that can **continuously display real-time date, time, and day** even during power failures. This project aims to address this issue by developing a **real-time clock system using an Arduino and DS3231 RTC module**, which maintains precise timekeeping with the help of a built-in battery backup and displays the information clearly on a **16x2 LCD**.

ABSTRACT

The Internet of Things (IoT) enables physical devices to communicate and share data in real time. This project presents a real-time date and day display system using an Arduino Uno, a DS3231 Real-Time Clock (RTC) module, and a 16x2 Liquid Crystal Display (LCD). The system continuously displays the current date, time, and day of the week on the LCD screen. The DS3231 module maintains accurate timekeeping even during power failures, thanks to its inbuilt battery backup. The Arduino communicates with both the RTC and LCD through the I2C communication protocol, ensuring efficient and reliable data transfer. This project demonstrates the practical implementation of IoT-based time display systems that can be used in smart clocks, attendance systems, scheduling devices, and other embedded applications requiring precise timekeeping.

INTRODUCTION

In today's technology-driven world, accurate timekeeping plays a crucial role in various real-time applications such as digital clocks, attendance systems, data loggers, and automation systems. The Internet of Things (IoT) enables devices to interact intelligently and share data efficiently, making real-time monitoring and control more reliable.

This project focuses on developing a real-time date and day display system using an Arduino Uno, a DS3231 Real-Time Clock (RTC) module, and a 16x2 Liquid Crystal Display (LCD). The system continuously displays the current date, time, and day on the LCD screen. The DS3231 module provides high precision and stability by maintaining accurate time even during power interruptions, thanks to its onboard battery backup.

The Arduino acts as the central controller, receiving time and date data from the RTC module through the I2C communication protocol and displaying it on the LCD. This project demonstrates the integration of hardware and software to achieve a reliable and efficient real-time clock display system. It serves as a foundation for more advanced IoT-based automation systems that require accurate timing and scheduling functionalities.

CONSTRUCTION AND WORKING

The multistage FIR rate conversion technique works on the principle of changing the sampling rate of a

1. Construction

The project is built around an **Arduino Uno** microcontroller, a **DS3231 Real-Time Clock (RTC) module**, and a **16x2 I2C LCD display**. These components are connected using the **I2C communication protocol**, which uses only two signal lines — **SDA (Serial Data)** and **SCL (Serial Clock)** — for data transfer between devices.

- The **DS3231 module** is connected to the Arduino through SDA (A4) and SCL (A5) pins. It receives 5V DC power and GND from the Arduino. The RTC has an inbuilt **backup battery**, which ensures that the time and date remain accurate even when the power supply is disconnected.
- The **16x2 I2C LCD** is also connected to the same SDA and SCL pins, sharing the I2C bus with the RTC module. It displays the current **time, date, and day** as received from the Arduino.
- The **Arduino Uno** acts as the main controller. It fetches the real-time data from the DS3231 module, processes it, and sends it to the LCD for display. The Arduino is powered via USB or an external 9V power source.
- A **breadboard** is used for interconnections, and jumper wires are used to connect all components neatly and securely.

This setup forms a compact, low-cost, and reliable system capable of displaying real-time information continuously.

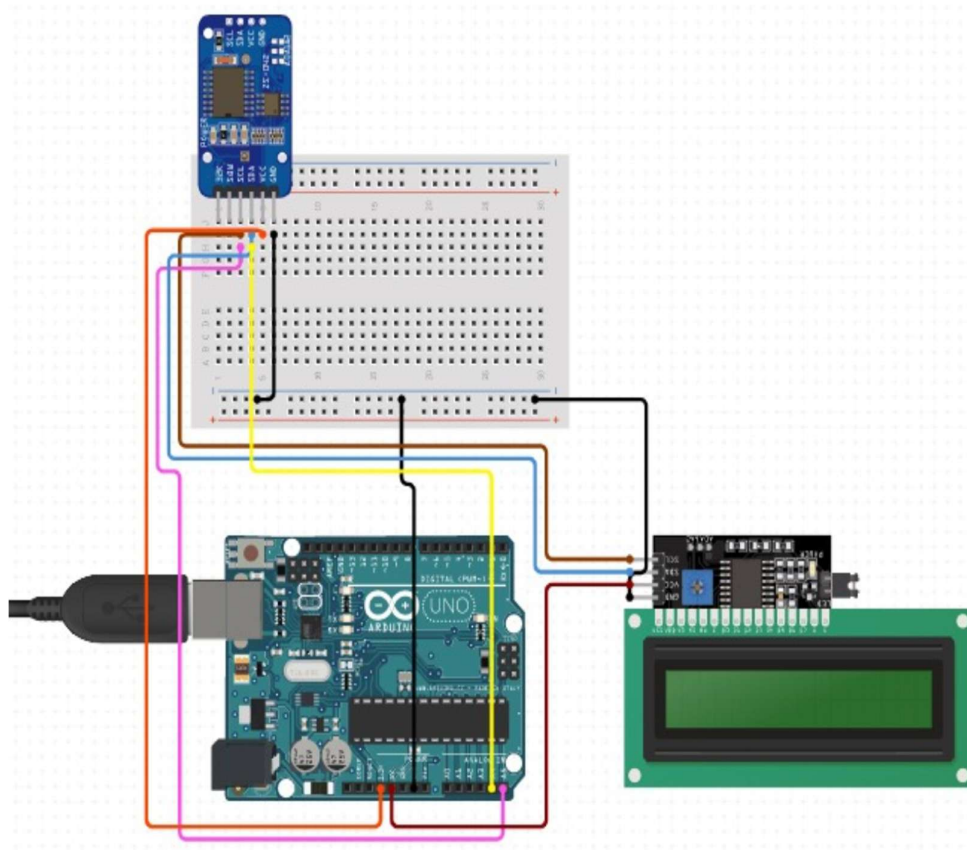
2. Working

1. When the system is powered ON, the Arduino initializes communication with both the RTC module and the LCD display through the I2C protocol.
2. The **DS3231 RTC module** continuously maintains and updates the current **time, date, and day** using its internal clock and crystal oscillator.
3. The **Arduino** requests this data from the RTC module at regular intervals.
4. The received data is then processed and formatted in a readable way (for example: 10-10-2025 | Friday | 01:25 PM).
5. The **formatted time and date** are sent to the **LCD**, which displays them clearly in real time.
6. Even if the main power supply is turned off, the RTC continues to keep track of the time using its **built-in coin cell battery**, ensuring accuracy when power is restored.
7. The system updates the LCD display every second, providing a continuous real-time clock functionality.

Key Features

- Accurate and automatic timekeeping.

- Continuous display of **date, time, and day**.
- Battery backup for uninterrupted time tracking.
- Uses only two communication lines (I2C) — reduces wiring complexity.
- Low-cost and efficient IoT-based embedded system.



ALGORITHM

Step 1: Start the Program and Include Required Libraries

Explanation:

To use the RTC and LCD modules, we first include the necessary libraries that provide functions for I2C communication, real-time clock handling, and LCD control.

Code Snippet:

```
#include <Wire.h>
#include <RTCLib.h>
#include <LiquidCrystal_I2C.h>
```

Step 2: Initialize RTC and LCD Objects

Explanation:

Create objects for the RTC module and the LCD display. These objects are used to call their respective library functions.

Code Snippet:

```
RTC_DS3231 rtc;
LiquidCrystal_I2C lcd(0x27, 16, 2);
```

Step 3: Define Days of the Week Array

Explanation:

The RTC returns the day as a number (0–6). We define an array to map each number to its corresponding weekday name.

Code Snippet:

```
const char *daysOfWeek[] = {
    "Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"
};
```

Step 4: Setup Function — Initialize Communication and Devices

Explanation:

In the `setup()` function, initialize I2C communication, the LCD, and check whether the RTC is connected properly.

Code Snippet:

```
void setup() {
    Wire.begin();
```

```
lcd.init();  
lcd.backlight();  
  
if (!rtc.begin()) {  
    lcd.print("RTC Not Found!");  
    while (1);  
}  
}
```

Step 5: Set RTC Time Manually (Optional, One-Time Setup)

Explanation:

The RTC must be set with the correct date and time once. After setting, it will continue keeping time accurately using its internal battery.

Code Snippet:

```
bool setRTCTimeOnce = true;  
  
if (setRTCTimeOnce) {  
    DateTime customTime(2025, 10, 7, 2, 24, 0);  
    rtc.adjust(customTime);  
    lcd.clear();  
    lcd.print("RTC Time Set!");  
    delay(2000);  
    setRTCTimeOnce = false;  
}
```

Step 6: Read Current Time and Date from RTC

Explanation:

Inside the loop(), we read the current time and date values from the DS3231 RTC using `rtc.now()`.

Code Snippet:

```
void loop() {  
    DateTime now = rtc.now();
```

Step 7: Display Day and Time on LCD (Line 1)

Explanation:

We use the first line of the LCD to show the current day and time (in hh:mm:ss format). Leading zeros are added for single-digit numbers to maintain proper formatting.

Code Snippet:

```
lcd.setCursor(0, 0);  
lcd.print("      ");  
lcd.setCursor(0, 0);  
  
lcd.print(daysOfWeek[now.dayOfTheWeek()]);  
lcd.print(" ");  
  
if (now.hour() < 10) lcd.print("0");  
lcd.print(now.hour());  
lcd.print(":");  
if (now.minute() < 10) lcd.print("0");  
lcd.print(now.minute());  
lcd.print(":");  
if (now.second() < 10) lcd.print("0");  
lcd.print(now.second());
```

Step 8: Display Date on LCD (Line 2)

Explanation:

The second line of the LCD displays the current date in dd/mm/yyyy format. Again, leading zeros are added where necessary.

Code Snippet:

```
lcd.setCursor(0, 1);  
lcd.print("      ");  
lcd.setCursor(0, 1);  
  
if (now.day() < 10) lcd.print("0");  
lcd.print(now.day());
```



```
lcd.print("/");  
if (now.month() < 10) lcd.print("0");  
lcd.print(now.month());  
lcd.print("/");  
lcd.print(now.year());
```

Step 9: Update Display Every Second

Explanation:

The program pauses for 1 second (`delay(1000)`) before reading new time values. This ensures the display updates in real time.

Code Snippet:

```
    delay(1000);  
}
```

Step 10: Continuous Loop

Explanation:

The `loop()` function repeats indefinitely, continuously fetching updated date and time values from the RTC and displaying them on the LCD.

INPUT CODE AND OUTPUT

```
#include <Wire.h>

#include <RTCLib.h>

#include <LiquidCrystal_I2C.h>


// Initialize RTC and LCD

RTC_DS3231 rtc;

LiquidCrystal_I2C lcd(0x27, 16, 2); // Use 0x3F if 0x27 doesn't work


// Array of days

const char *daysOfWeek[] = {

    "Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"

};


// Flag to set RTC time manually once

bool setRTCtimeOnce = true;


void setup() {

    Wire.begin();

    lcd.init();

    lcd.backlight();


    // Check RTC

    if (!rtc.begin()) {

        lcd.print("RTC Not Found!");

        while (1); // Stop execution

    }

}
```

```

}

// Set RTC time manually (only run once)
if (setRTCTimeOnce) {
    DateTime customTime(2025, 10, 7, 2, 24, 0); // YYYY, MM, DD, HH, MM, SS
    rtc.adjust(customTime);

    lcd.clear();
    lcd.print("RTC Time Set!");
    delay(2000);

    // Disable flag after first set
    setRTCTimeOnce = false;
}

lcd.clear();
}

void loop() {
    DateTime now = rtc.now(); // Get current RTC time

    // --- Line 1: Day + Time ---
    lcd.setCursor(0, 0);
    lcd.print("          "); // Clear line
    lcd.setCursor(0, 0);

    lcd.print(daysOfWeek[now.dayOfTheWeek()]);
    lcd.print(" ");

```

```

if (now.hour() < 10) lcd.print("0");
lcd.print(now.hour());
lcd.print(":");
if (now.minute() < 10) lcd.print("0");
lcd.print(now.minute());
lcd.print(":");
if (now.second() < 10) lcd.print("0");
lcd.print(now.second());

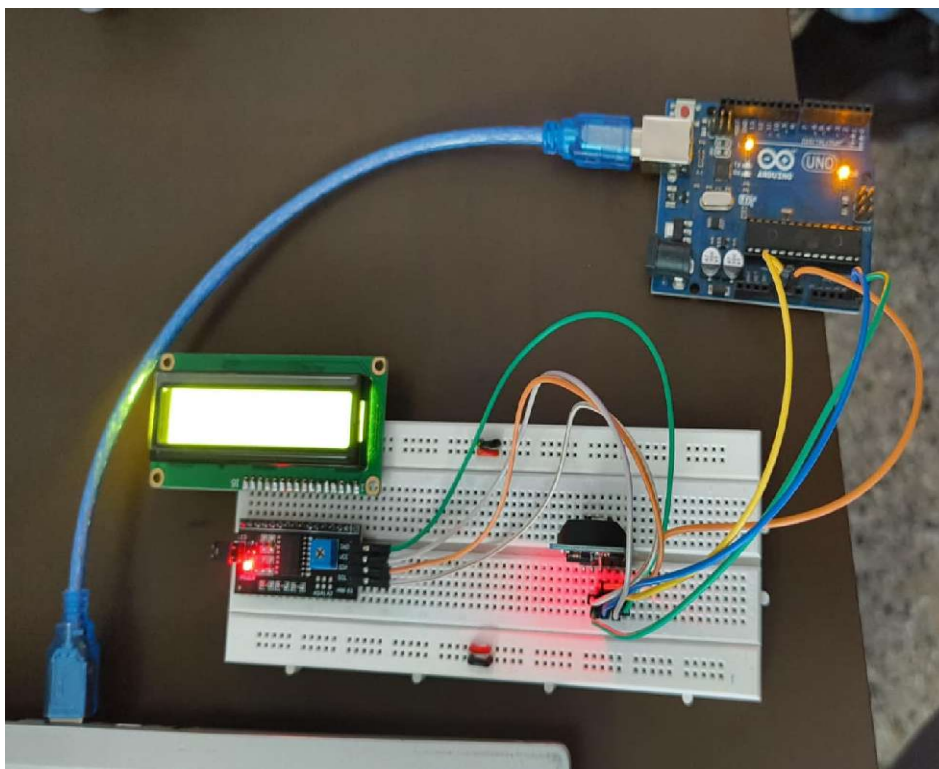
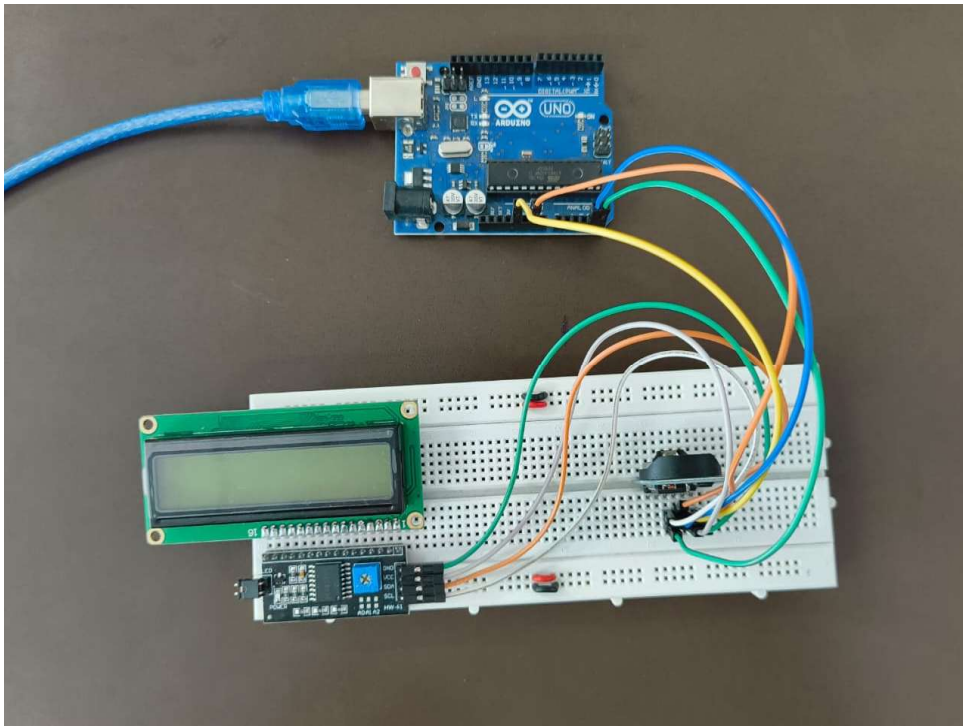
// --- Line 2: Date ---
lcd.setCursor(0, 1);
lcd.print("          "); // Clear line
lcd.setCursor(0, 1);

if (now.day() < 10) lcd.print("0");
lcd.print(now.day());
lcd.print("/");
if (now.month() < 10) lcd.print("0");
lcd.print(now.month());
lcd.print("/");
lcd.print(now.year());

delay(1000); // Update every second
}

```

OUTPUT :



APPLICATIONS.

1. Digital Clocks and Wall Displays

- Can be used in homes, offices, and schools to show accurate real-time date, day, and time.

2. Attendance and Access Control Systems

- Helps in logging employee or student attendance automatically with precise timestamps.

3. Data Logging Systems

- Useful in recording time-stamped data for temperature, humidity, or other sensors in IoT projects.

4. Automation Systems

- Can trigger tasks or appliances at specific times, integrating with smart home or industrial automation.

5. Scheduling Devices

- Ideal for timers, reminders, or alarm systems requiring accurate date and time.

6. IoT-Based Monitoring Systems

- Enables synchronization of events across multiple devices in real-time applications.

7. Educational and Demonstration Purposes

- Serves as a simple embedded system project for learning Arduino, RTC modules, and I2C communication.

REFERENCES

1. **Arduino Official Documentation** – Arduino Uno, <https://www.arduino.cc/en/Guide/ArduinoUno>
2. **DS3231 RTC Module Datasheet** – Maxim Integrated, <https://www.maximintegrated.com/en/products/analog/real-time-clocks/DS3231.html>
3. **LiquidCrystal_I2C Library Documentation** – https://www.arduino.cc/reference/en/libraries/liquidcrystal_i2c/
4. Muhammad Ali Mazidi, **“PIC Microcontroller and Embedded Systems”**, Pearson Education, 2008.
5. Dogan Ibrahim, **“Advanced PIC Microcontroller Projects in C”**, Elsevier, 2014.
6. Prof. Shalini D. et al., **“Real-Time Clock Interfacing with Arduino for Embedded Systems”**, International Journal of Engineering Research & Technology (IJERT), Vol. 6, Issue 4, 2017.